



Directory > ... >
Serverless ETL pipelines

 Copy page



Serverless ETL pipelines

Last reviewed: over 1 year ago

Introduction

Extract, Transform, Load (ETL) pipelines are a cornerstone in the realm of data engineering, facilitating the seamless flow of data from its raw state to a structured, usable format. ETL pipelines are instrumental in the data processing journey, particularly in scenarios where data needs to be collected, cleansed, and transformed before being loaded into a target destination.

The process begins with extraction, where data is gathered from various sources such as databases, files, or streams. This raw data is often disparate and unstructured, necessitating the next step: transformation. During transformation, the data undergoes a series of operations to standardize formats, clean inconsistencies, and enrich with additional context or calculations. This phase is critical for ensuring data quality and consistency, as well as aligning it with the requirements of downstream applications and analytics.

Finally, the transformed data is loaded into a destination, which could be a data warehouse, database, or any other storage solution. The loading phase involves efficiently moving the processed data to its intended destination, where it can be readily accessed and utilized for various purposes such as reporting, analysis, or feeding into machine learning models.

ETL pipelines play a pivotal role in data-driven decision-making processes across industries, enabling organizations to derive insights and value from their data assets. By automating and streamlining the journey from raw data to actionable insights, ETL pipelines empower businesses to make informed decisions, optimize processes, and gain competitive advantages in today's data-driven landscape.

Examples of ETL pipelines in action include scenarios like extracting sales data from multiple retail stores, transforming it to a standardized format, and loading it into a centralized data warehouse for analysis and reporting purposes. Similarly, ETL pipelines are utilized in data migration projects, where legacy data needs to be migrated to modern systems while ensuring data integrity and consistency throughout the process.

Cloudflare allows for the deployment of fully serverless ETL pipelines, which can reduce complexity, time to production and overall cost. The following diagrams

demonstrate different methods of how Cloudflare can be used in common ETL pipeline deployments.

ETL pipeline with HTTP-based ingest

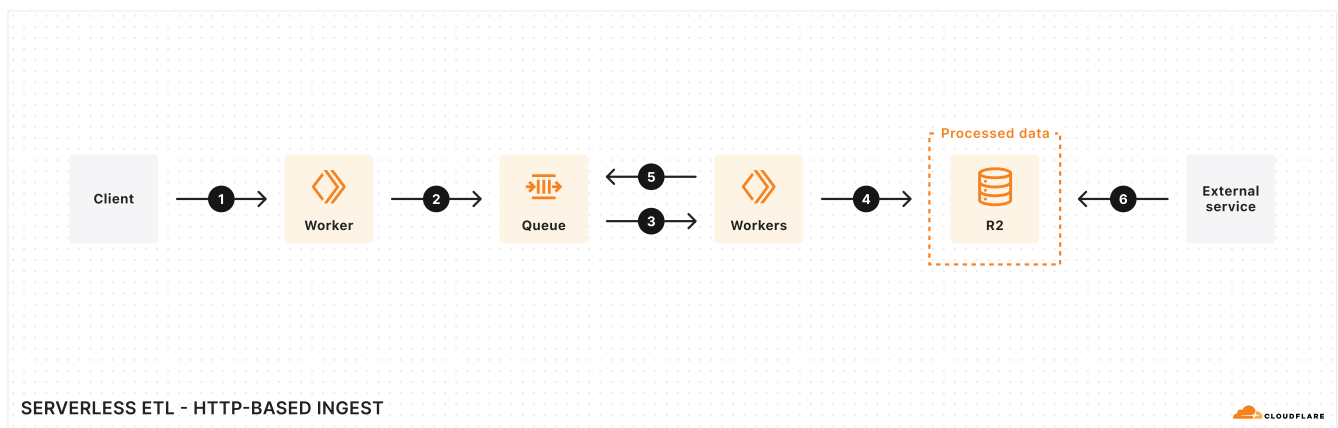


Figure 1: ETL pipeline with HTTP-based ingest

This architecture shows a fully serverless ETL pipeline with an API endpoint as ingest. Clients send data via HTTP request to be processed. Common examples include click-stream data or analytics.

1. **Client request:** Send POST request with data to be ingested. Examples would include click-stream data, analytics endpoints.

2. **Input processing:** Process incoming request using [Workers](#) and send messages to [Queues](#) to add to processing backlog.
3. **Data processing:** Use [Queues](#) to trigger a [consumer](#) that process input data in batches to prevent downstream overload and increase efficiency. The consumer performs all data cleaning, transformation and standardization operations.
4. **Object storage:** Upload processed data to [R2](#) for persistent storage.
5. **Ack/Retry mechanism:** Signal success/error by using the [Queues Runtime API](#) in the consumer for each document. [Queues](#) will schedule retries, if needed.
6. **Data querying:** Access processed data from external services for further data usage.

ETL pipeline with object storage ingest

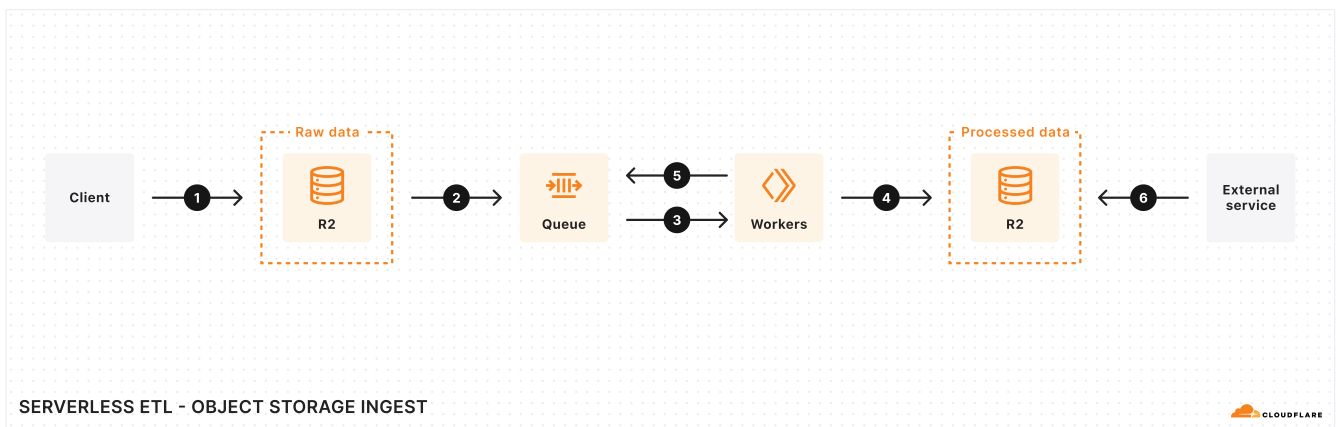


Figure 2: ETL pipeline with object storage ingest

This architecture shows a fully serverless ETL pipeline with object storage as ingest. Common examples include log and unstructured document processing.

1. **Client request:** Upload raw data to R2 via S3-compatible API. Common examples include log and analytics data.
2. **Input processing:** Send messages to [Queues](#) using [R2 event notifications](#) upon object upload.
3. **Data processing:** Use [Queues](#) to trigger a [consumer](#) that process input data in batches to prevent downstream overload and increase efficiency. The consumer performs all data cleaning, transformation and standardization operations.

4. **Object storage:** Upload processed data to [R2](#) for persistent storage.
5. **Ack/Retry mechanism:** Signal success/error by using the [Queues Runtime API](#) in the consumer for each document. [Queues](#) will schedule retries, if needed.
6. **Data querying:** Access processed data from external services for further data usage.

Related resources

- [Workers: Get started](#)
- [Queues: Get started](#)
- [R2: Get started](#)

Was this helpful?



[Previous](#)[Next](#)[Programmable
Platforms](#)[Serverless global
APIs](#)

Last updated: Aug 19, 2025

Resources

[API](#)[New to Cloudflare?](#)[Directory](#)[Sponsorships](#)[Open Source](#)

Support

[Help Center](#)[System Status](#)[Compliance](#)[GDPR](#)

Company

[cloudflare.com](#)[Our team](#)[Careers](#)

Tools

[Cloudflare Radar](#)[Speed Test](#)[Is BGP Safe Yet?](#)[RPKI Toolkit](#)[Certificate Transparency](#)

Community



© 2025 Cloudflare, Inc. • [Privacy Policy](#) • [Terms of Use](#)

• [Report Security Issues](#) • [Trademark](#) •  [Cookie Preferences](#)